Instrumentos Computacionales - Stata

Maestría en Economía

Santiago Cerutti

UNLP

Marzo 2026

Contenido - Clase 3

1. Matrices
2. Funciones macro extendidas
3. Listas de variables
4. Estimates store y coefplot
5. Proyectos
6. Múltiples do-files
7. Replicabilidad

Contenido - Clase 3

1. Matrices

2. Funciones macro extendidas

3. Listas de variables

4. Estimates store y coefplot

5. Proyectos

6. Múltiples do-files

7. Replicabilidad

Matrices

Solemos usarlas para acumular y visualizar resultados, pero también es posible realizar operaciones entre matrices.

Sintaxis para crear una matriz:

```
matrix [define] matrix_name = matrix_expression
```

Matrices

```
* Ejemplo para crear una matriz de una sola fila y 3 columnas
matrix define A = (1 , 2 , 3)

* Ejemplo para crear una matriz de 3 filas y 3 columnas
matrix B = (1 , 2 , 3 \ 4 , 5 , 6 \ 7 , 8 , 9)
```

Matrices

Para ver el contenido, usamos el comando **`matrix list`**:

```
mat list A
```

```
A[1,3]
    c1  c2  c3
r1   1   2   3
```

```
mat list B
```

```
B[3,3]
    c1  c2  c3
r1   1   2   3
r2   4   5   6
r3   7   8   9
```

Matrices

Una alternativa para crear una matriz:

```
matrix [define] matrix_name = J(#1 , #2 , #3)
```

Donde:

- #1: número de filas
- #2: número de columnas
- #3: número para rellenar todas las celdas de la matriz

```
* Ejemplo para crear una matriz de 4 filas , 2 columnas y todas sus celdas
    iguales a 0
matrix define C = J(4 , 2 , 0)
```

Matrices

También podemos definir valores específicos de alguna celda. Por ejemplo, para cambiar el contenido de la celda correspondiente a la primer fila y segunda columna por un 9:

```
          nombre matriz
                |
matrix define C[ 1  ,  2] = 9
                  |    |
                 fila  columna
```

Matrices

Por lo tanto, también es posible ir alterando sistemáticamente valores de una matriz a lo largo de todas sus celdas utilizando bucles.

Sin bucle

```
matrix define B[1 , 1] = 10
matrix define B[2 , 1] = 20
matrix define B[3 , 1] = 30
matrix define B[4 , 1] = 40
```

Con bucle

```
forvalues i = 1/4 {
    matrix define B[`i' , 1] = `i'*10
}
```

Matrices

Ejercicio

Crear con un bucle anidado esta matriz:

```
B[3,4]
    c1  c2  c3  c4
r1   1   2   3   4
r2   5   6   7   8
r3   9  10  11  12
r4  13  14  15  16
```

Matrices

Otra alternativa para crear matrices es a partir de los resultados de un comando *rclass*:

```
* tabstat con save guarda resultados en r(StatTotal)
tabstat price mpg weight , stat(mean sd min max) save

* Creo una matriz a partir del resultado
matrix define M = r(StatTotal)

* Visualizo la matriz creada
matrix list M
```

```
M[4,3]
          price        mpg     weight
Mean   6165.2568  21.297297  3019.4595
  SD   2949.4959  5.7855032  777.19357
 Min        3291         12       1760
 Max       15906         41       4840
```

Matrices

Otros comandos útiles relacionados con matrices:

- Cambiar los nombres de las filas o columnas (**`mat rowname`** o **`mat colname`** respectivamente)
- Agregar el contenido de una matriz a la base de datos **`svmat`**
- Crear matrices directamente desde la base de datos **`mkmat`**

Contenido - Clase 3

1. Matrices

2. Funciones macro extendidas

3. Listas de variables

4. Estimates store y coefplot

5. Proyectos

6. Múltiples do-files

7. Replicabilidad

Funciones macro extendidas

- Además de permitir expresiones, las macro locales y globales permiten el uso de funciones extendidas para tareas como: extraer atributos de una variable, dataset o parámetros del sistema, detectar archivos, entre otras.
- El uso de una función macro extendida se denota mediante el caracter "**:**" luego del nombre de la macro.

```
local macname : ...
```

Funciones macro extendidas

Ejemplo:

```
sysuse auto , clear
local etiqueta : var label price
local tipo : type price

display "La etiqueta de la variable price es: `etiqueta'"
display "El tipo de la variable price es: `tipo'"
```

Funciones macro extendidas

Otros ejemplos (hay más)

Funcion	Devuelve
`value label varname`	el nombre de la etiqueta de valor asociada a *varname*.
`label (varname) #`	la etiqueta de valor de # en *varname*.
`variable label varname`	la etiqueta asociada a *varname*.
`sortedby`	los nombres de las variables por las que actualmente se ordenan los datos en memoria.
`rownames matname`	los nombres de las filas de la matriz *matname*.
`colnames matname`	los nombres de las columnas de la matriz *matname*.

Funciones macro extendidas

Una muy útil para cuando trabajamos con muchos archivos:

```
* Almacenar todos los nombres de los archivos Excel de una carpeta

 local archivos : dir "ruta de la carpeta" files "*.xlsx"
```

Lo mismo podemos hacer con cualquier otra extensión. Esto resulta muy útil para iterar entre archivos sin la necesidad de saber sus nombres.

Funciones macro extendidas

También suelen ser útiles dentro de los bucles:

```
foreach v of varlist price mpg weight {
    local etiqueta : variable label `v'
    kdensity `v' , title("Distribucion de `etiqueta'")
}
```

Lo mismo podemos hacer con cualquier otra extensión. Esto resulta muy útil para iterar entre archivos sin la necesidad de saber sus nombres.

Contenido - Clase 3

1. Matrices

2. Funciones macro extendidas

3. Listas de variables

4. Estimates store y coefplot

5. Proyectos

6. Múltiples do-files

7. Replicabilidad

Listas de variables

- Para poder iterar entre variables, a menudo es necesario construir listados para almacenarlas en macros y reutilizarlas posteriormente.
- Dos comandos muy útiles para definir estas listas son **ds** y **unab**.

Sintaxis:

```
unab lmacname : [varlist] [ , options]
```

```
ds [varlist] [ , options]
```

Listas de variables

Diferencias

- Ambos permiten construir listas de variables para almacenarlas en macros y reutilizarlas.:
  - **ds** selecciona variables por criterios (tipo, formato, nombre, etc.) y devuelve la lista en `r(varlist)`.
  - **unab** expande una *varlist* abreviada y la guarda directamente en una macro definida por el usuario.

Listas de variables

Ventajas y desventajas

- **ds** es flexible para filtros inteligentes. Pero `r(varlist)` se sobreescribe con el siguiente comando *rclass* que ejecutemos.
- **unab** deja una lista estable sin depender de `r()` y permite usar comodines para expandir variables con patrones comunes (por ejemplo, x* para todas las variables que comienzan con x). Pero no filtra por atributos.

Listas de variables

Ejemplo ds y unab

```
sysuse auto , clear

* Selecciono variables numericas con ds
ds , has(type numeric)
local num_ds `r(varlist)'

* Guardo una varlist con unab
unab a: tr *t

* Muestro ambos listados
display "Con ds:   `num_ds'"
display "Con unab: `a'"
```

Contenido - Clase 3

1. Matrices

2. Funciones macro extendidas

3. Listas de variables

4. Estimates store y coefplot

5. Proyectos

6. Múltiples do-files

7. Replicabilidad

Estimates store

- El comando **`estimates store`** guarda en memoria los resultados del último modelo estimado.
- Esto permite nombrar cada especificación y recuperarla después para comparar modelos.
- Es útil cuando queremos:
  - armar tablas comparativas (por ejemplo con **`estimates table`**),
  - graficar coeficientes de varios modelos (por ejemplo con **`coefplot`**),

Estimates store

Flujo de trabajo típico

```
* 1) Estimo un modelo
regress price mpg weight

* 2) Guardo sus resultados con el nombre "M1"
estimates store M1

* 3) Estimo otra especificacion
regress price mpg weight foreign

* 4) Guardo este modelo con el nombre "M2"
estimates store M2
```

Estimates store

Ejemplo completo

```
sysuse auto , clear

* Modelo sin controles adicionales
regress price mpg weight
estimates store M1

* Modelo agregando foreign
regress price mpg weight foreign
estimates store M2

* Comparo ambos modelos en una tabla
estimates table M1 M2 , b(%9.2f) se stats(N r2)
```

Coefplot

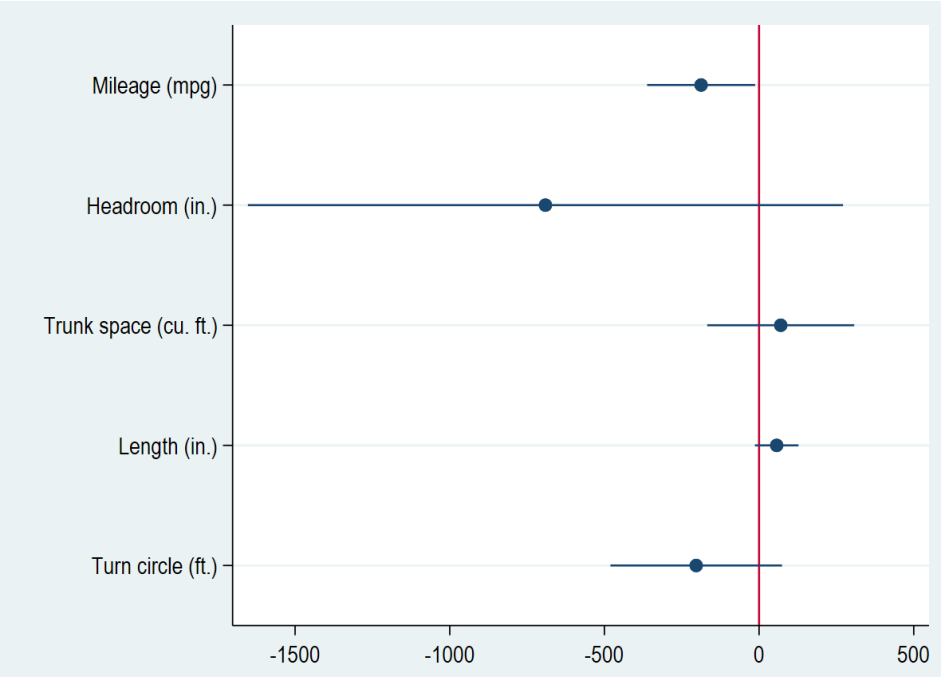
- El comando **`coefplot`** permite graficar coeficientes de regresión de manera visual y comparativa.
- Es especialmente útil para presentar resultados de múltiples modelos o especificaciones en un solo gráfico.
- Facilita la interpretación tanto de las magnitudes como de la significancia estadística de los coeficientes en comparación con verlos en una tabla.

Coefplot

- Para utilizar **`coefplot`**, primero debemos instalar el paquete correspondiente utilizando el comando **`ssc install coefplot`**.
- Luego, después de ejecutar nuestras regresiones y almacenarlas con **`estimates store`**, podemos usar **`coefplot`** para graficar los modelos y coeficientes de interés.

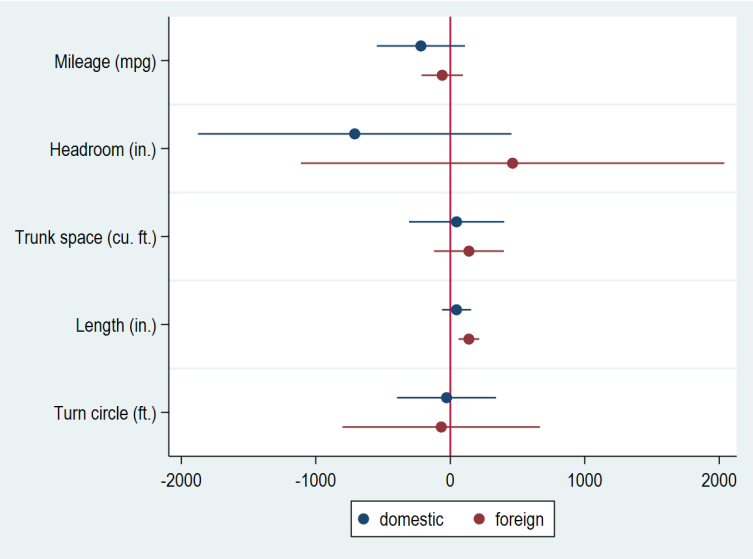
Ejemplo 1

```
sysuse auto
regress price mpg headroom trunk length turn
coefplot , drop(_cons) xline(0)
```



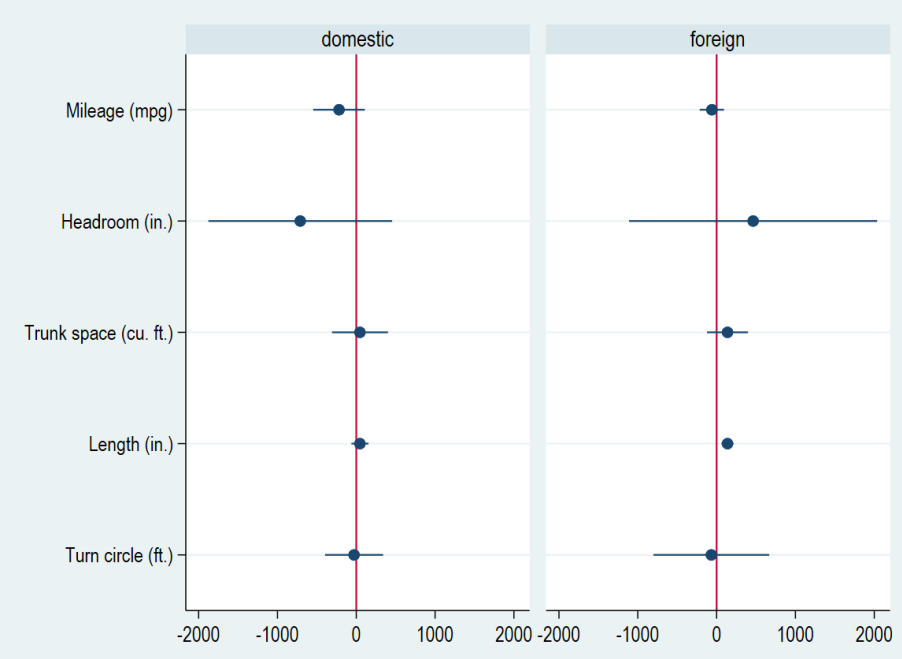
Ejemplo 2

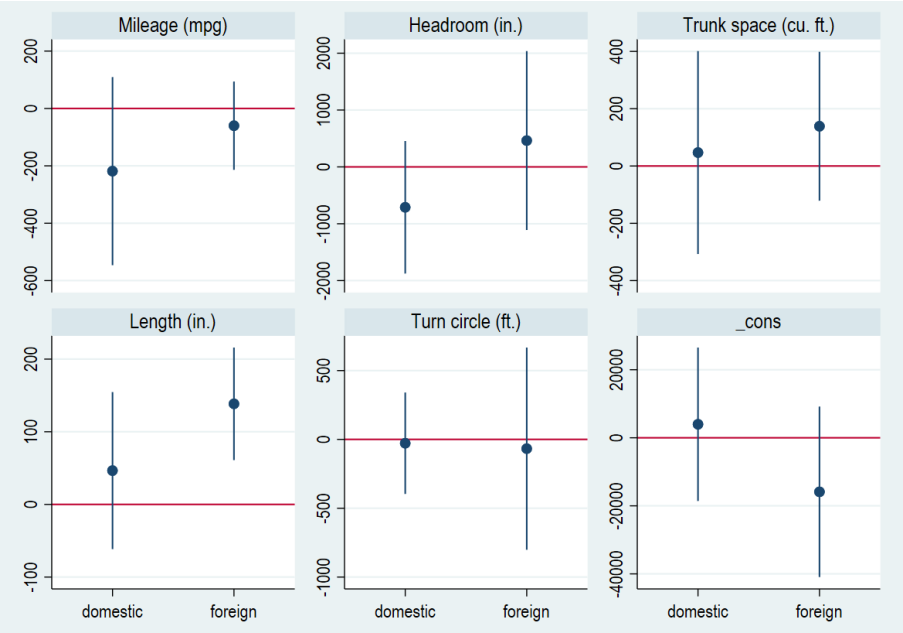
```
regress price mpg headroom trunk length turn if foreign==0
estimates store domestic

regress price mpg headroom trunk length turn if foreign==1
estimates store foreign

coefplot domestic foreign , drop(_cons) xline(0)
```



Ejemplo 3

```
coefplot domestic || foreign , drop(_cons) xline(0)

coefplot domestic || foreign , yline(0) bycoefs vertical byopts(yrescale)
```





Coefplot

En esta página pueden encontrar más ejemplos de personalización de gráficos con **`coefplot`**:

`https://repec.sowi.unibe.ch/stata/coefplot/getting-started.html`

Exportar tablas con esttab

- **`esttab`** (del paquete **`estout`**) permite exportar resultados de regresión en formato de tabla.
- Es útil para generar tablas listas para informes en Word, Excel o LaTeX.

Exportar tablas con esttab

Ejemplo básico

```
* Instalar estout (si no esta instalado)
ssc install estout

sysuse auto , clear

regress price mpg weight
estimates store M1

regress price mpg weight foreign
estimates store M2

* Tabla en consola
esttab M1 M2 , se star(* 0.10 ** 0.05 *** 0.01)
```

Exportar tablas con esttab

```
* Exporto a CSV (apto para Excel)
esttab M1 M2 using "../../tables/regresiones_auto.csv" , ///
      replace se star(* 0.10 ** 0.05 *** 0.01)

* Exporto a LaTeX
esttab M1 M2 using "../../tables/regresiones_auto.tex" , ///
      replace se label title("Modelos para price")
```

	A	B	C	D
1		M1	M2	
2		b/se	b/se	
3	mpg	-49.51222	21.8536	
4		86.15604	74.22114	
5	weight	1.746559***	3.464706***	
6		0.6413538	0.630749	
7	foreign		3673.06***	
8			683.9783	
9	_cons	1946.069	-5853.696*	
10		3597.05	3376.987	
11	N	74	74	
12				

Table 1: Modelos para price

	(1) Price	(2) Price
Mileage (mpg)	-49.51	21.85
	(86.16)	(74.22)
Weight (lbs.)	1.747**	3.465***
	(0.641)	(0.631)
Car origin		3673.1***
		(684.0)
Constant	1946.1	-5853.7
	(3597.0)	(3377.0)
Observations	74	74

Standard errors in parentheses

* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Ejemplo integrador

```
sysuse auto , clear

* Defino lista base de regresores (solo unab)
unab base : mpg weight

* Armo varias especificaciones
local spec1 "`base'"
local spec2 "`base' length turn"
local spec3 "`base' length turn foreign"

* Estimo y guardo cada especificacion en un bucle
estimates clear
forvalues i = 1/3 {
    regress price `spec`i''
    estimates store M`i'
}
```

Ejemplo integrador (continuación)

```
* Comparo coeficientes de los modelos guardados
coefplot (M1 , label("Esp. 1")) ///
         (M2 , label("Esp. 2")) ///
         (M3 , label("Esp. 3")) , ///
    keep(mpg weight length turn ) drop(_cons) xline(0) ///
    title("Coeficientes")

* Exporto el gráfico
graph export "../../figures/coefplot_integrador.png" , replace

* Exporto tabla con esttab
esttab M1 M2 M3 using "../../tables/ejemplo_integrador.tex" , ///
    replace se label mtitles("Esp. 1" "Esp. 2" "Esp. 3") ///
    title("Especificaciones para price")
```

Contenido - Clase 3

1. Matrices

2. Funciones macro extendidas

3. Listas de variables

4. Estimates store y coefplot

5. Proyectos

6. Múltiples do-files

7. Replicabilidad

Proyectos

Rutas

- Cuando trabajamos en varias PC o con otras personas es muy importante que la definición de las rutas sea clara y fácilmente adaptable en el do-file.
- Mientras menos líneas de código tengamos que modificar para adaptar el do-file, mejor.

Proyectos

Rutas

- Veamos en un ejemplo de un proyecto simple con solo 3 archivos y carpetas. Existen varias formas de definir las rutas y sus ventajas y desventajas.

```
Documents/
├── Code/
│   └── codigo.do
├── Bases/
│   └── base1.dta
└── Tablas/
    └── tabla1.xls
```

Proyectos

Rutas

Ejemplo 1: Utilizando rutas completas

```
* Abro la base
 use "C:\Users\Documents\Bases\base1.dta"

  ... una serie de comandos ...

* Guardo la base y exporto la tabla
  save "C:\Users\Documents\Bases\base1_modificada.dta"
  export excel "C:\Users\Documents\Tablas\tabla1.xls"
```

Proyectos

Rutas

Ejemplo 1: Utilizando rutas completas

```
* Abro la base
 use "C:\Users\Documents\Bases\base1.dta"

  ... una serie de comandos ...

* Guardo la base y exporto la tabla
  save "C:\Users\Documents\Bases\base1_modificada.dta"
  export excel "C:\Users\Documents\Tablas\tabla1.xls"
```

Pregunta

¿Cuántas líneas y cuáles debería modificar para utilizar el do en otra PC?

Proyectos

Rutas

Ejemplo 2: Definiendo **cd** al inicio del do-file

```
* Defino el directorio de trabajo
  cd "C:\Users\Documents"

* Abro la base
  use "Bases\base1.dta"

  ... una serie de comandos ...

* Guardo la base y exporto la tabla
  save "Bases\base1_modificada.dta"
  export excel "Tablas\tabla1.xls"
```

Proyectos

Rutas

Ejemplo 2: Definiendo **cd** al inicio del do-file

```
* Defino el directorio de trabajo
  cd "C:\Users\Documents"

* Abro la base
  use "Bases\base1.dta"

  ... una serie de comandos ...

* Guardo la base y exporto la tabla
  save "Bases\base1_modificada.dta"
  export excel "Tablas\tabla1.xls"
```

Pregunta

¿Cuántas líneas y cuáles debería modificar para utilizar el do en otra PC?

Proyectos

Rutas

Ejemplo 3: Utilizando macros locales

```
* Defino las rutas en macros locales
  local ruta_bases = "C:\Users\Documents\Bases"
  local ruta_tablas = "C:\Users\Documents\Tablas"

* Abro la base
  use "`ruta_bases'\base1.dta"

  ... una serie de comandos ...

* Guardo la base y exporto la tabla
  save "`ruta_bases'\base1_modificada.dta"
  export excel "`ruta_tablas'\tabla1.xls"
```

Proyectos

Rutas

Ejemplo 4: Utilizando macros globales

```
* Defino las rutas en macros globales
  global ruta_bases = "C:\Users\Documents\Bases"
  global ruta_tablas = "C:\Users\Documents\Tablas"

* Abro la base
  use "${ruta_bases}\base1.dta"

  ... una serie de comandos ...

* Guardo la base y exporto la tabla
  save "${ruta_bases}\base1_modificada.dta"
  export excel "${ruta_tablas}\tabla1.xls"
```

Proyectos

Rutas

Ejemplo 5: Definiendo rutas y nombres de archivos.

```
* Defino las rutas en macros locales
  local ruta_bases = "C:\Users\Documents\Bases"
  local ruta_tablas = "C:\Users\Documents\Tablas"

* Defino los nombres de los archivos en macros locales
  local archivo_base = "base1.dta"
  local archivo_base_modificada = "base1_modificada.dta"
  local archivo_tabla = "tabla1.xls"

* Abro la base
  use "`ruta_bases'\`archivo_base'"

  ... una serie de comandos ...

* Guardo la base y exporto la tabla
  save "`ruta_bases'\`archivo_base_modificada'"
  export excel "`ruta_tablas'\`archivo_tabla'"
```

Proyectos

Rutas

OJO CON LAS LOCALES Y GLOBALES SEPARADAS POR BARRAS.

```
local a = "nombre_carpeta"
local b = "nombre_archivo"

* Con \ es necesario usarla 2 veces
di "`a'\`b'"
di "`a'\\`b'"

* Con / se usa 1 vez
di "`a'/`b'"
di "`a'//`b'"
```

```
nombre_carpeta`b'
nombre_carpeta\nombre_archivo

nombre_carpeta/nombre_archivo
nombre_carpeta//nombre_archivo
```

Proyectos

Rutas

OJO CON LAS LOCALES Y GLOBALES SEPARADAS POR BARRAS.

```
global a = "nombre_carpeta"
global b = "nombre_archivo"

di "$a\$b"        * No va a tomar la segunda global
di "${a}\\${b}"   * Toma 1 barra \
di "$a/$b"        * Toma 1 barra /
di "${a}/${b}"    * Toma 1 barra /
```

```
nombre_carpeta$b
nombre_carpeta\nombre_archivo
nombre_carpeta/nombre_archivo
nombre_carpeta//nombre_archivo
```

Contenido - Clase 3

1. Matrices

2. Funciones macro extendidas

3. Listas de variables

4. Estimates store y coefplot

5. Proyectos

6. Múltiples do-files

7. Replicabilidad

Proyectos

Múltiples do-files

- Hasta ahora siempre trabajamos con un único do-file, que al ejecutarlo resolvía todas las tareas necesarias.
- Sin embargo, para proyectos extensos esto no suele ser lo más conveniente.
- Una solución consiste en agrupar y separar en distintos do-files estas tareas según las funciones que realizan en el proyecto:

DO 1	DO 2	DO 3
preapara.do	regresiones.do	graficos.do

Proyectos

Múltiples do-files

- Sin embargo, también es recomendable poder replicar todas las acciones del proyecto sin tener que estar ejecutando cada do-file individualmente.
- Para esto podemos crear un do-file exclusivo para ejecutar todos los do-files del proyecto, en el orden que corresponda.
- A este do-file suele llamársele **master**.

Múltiples do-files

- De esta manera, cada do-file se encarga de una función específica dentro del proyecto.
- El do-file **master** se encargaría de ejecutar los 3 do-files, en el orden correcto usando el comando **do**.
- Una alternativa, es incluirlos en el archivo master utilizando el comando **include**.

Proyectos

Ejemplo archivo Master

```
* master.do
version 18
clear all
set more off

* Ejecuto los do-files del proyecto en orden
do prepara.do
do regresiones.do
do graficos.do
```

Contenido - Clase 3

1. Matrices

2. Funciones macro extendidas

3. Listas de variables

4. Estimates store y coefplot

5. Proyectos

6. Múltiples do-files

7. Replicabilidad

Proyectos

Replicabilidad

Ejemplo de un proyecto con una organización clara de archivos y carpetas y con un do-file master que ejecuta todo el flujo.

```
Proyecto/
├── code/
│   ├── 01_prepara.do
│   ├── 02_regresiones.do
│   └── 03_graficos.do
├── bases/
│   ├── raw/
│   │   └── base_1.xlsx
│   └── processed/
│       └── base_final.dta
├── graficos/
│   └── grafico_1.png
├── tablas/
│   └── tabla1.xls
└── master.do
```

Proyectos

Replicabilidad

Ejemplo de archivo **`master.do`**: para esta estructura:

```
* master.do (ubicado en la carpeta Proyecto)
version 18
clear all
set more off

* Defino rutas
global ROOT      "C:/ruta_ejemplo/Proyecto" // raíz del proyecto
global RAW       "${ROOT}/bases/raw"        // insumos originales
global PROCESSED "${ROOT}/bases/processed"  // bases limpias y analíticas
global GRAFICOS  "${ROOT}/graficos"         // gráficos
global TABLAS    "${ROOT}/tablas"           // tablas

* Ejecuto dofiles en orden
do "${ROOT}/code/01_prepara.do"
do "${ROOT}/code/02_regresiones.do"
do "${ROOT}/code/03_graficos.do"
```

Proyectos

Replicabilidad

Ejemplo de archivo **`01_prepara.do`**:

```
* 01_prepara.do
* Importa y limpia los datos crudos

* Importo datos crudos
import delimited "${RAW}/ventas.csv", clear

... Más comandos ...

* Guardo base procesada
save "${PROCESSED}/ventas_limpia.dta", replace
```

Proyectos

Replicabilidad

Ejemplo de archivo **`01_prepara.do`**:

```
* 01_prepara.do
* Importa y limpia los datos crudos

* Importo datos crudos
import delimited "${RAW}/ventas.csv", clear

... Más comandos ...

* Guardo base procesada
save "${PROCESSED}/ventas_limpia.dta", replace
```

Pregunta

¿Cuántas líneas y cuáles debería modificar para utilizar el do en otra PC?

Proyectos

Replicabilidad

Pregunta

¿Es posible no tener que cambiar ninguna línea??

Proyectos

```
* Ejemplo proyectos - Master
clear all

*/ Ruta Raiz
* Desktop
capture cd "F:/Docencia/Instrumentos-computacionales/Seminario-Stata-MEFCE-2026/code/ejemplos/proyectos"
* Notebook
capture cd "C:/Users/wb588478/Documents/personal_code/Seminario-Stata-MEFCE-2026/ejemplos/proyectos"

*/ Rutas proyecto
global ROOT = c(pwd)
global CODE      "${ROOT}/code"
global RAW       "${ROOT}/data/raw"
global PROCESSED "${ROOT}/data/processed"
global TABLES    "${ROOT}/tables"
global GRAPHS    "${ROOT}/graphs"

*/ Crear carpetas
cap mkdir "${RAW}"
cap mkdir "${PROCESSED}"
cap mkdir "${TABLES}"
cap mkdir "${GRAPHS}"

*/ Ejecutar los dofiles
do "${CODE}/01_prepara.do"
do "${CODE}/02_analisis.do"
do "${CODE}/03_reporte_graficos.do"

*/ End
```

Proyectos

Replicabilidad

Checklist mínimo para que un proyecto sea fácilmente replicable:

- Carpeta del proyecto ordenada y con nombres consistentes.
- Un único do-file **master** que ejecute todo el flujo.
- Rutas definidas al inicio (evitar hardcodear rutas en todo el código).
- Data de entrada claramente identificada (raw).
- Productos separados por tipo: bases, tablas, gráficos y logs.

Proyectos

Replicabilidad

Buenas prácticas de ejecución:

- Documentación: Incluir un **README.md** con requisitos, estructura y orden de ejecución.
- Comentar supuestos y decisiones de limpieza de datos.
- Usar control de versiones para registrar cambios en do-files.
- Mantener nombres descriptivos y estables para variables y archivos.
- Separar datos **raw** de datos **procesados** para preservar trazabilidad.

Ejercicio Optativo

- Proponer alguna sugerencia o corrección en algún do-file dentro de los ejercicios optativos subido al repositorio por algún compañero.
- La idea es ejercitar la lectura de código y la colaboración en proyectos utilizando GitHub.

Otros temas interesantes

Para el que se haya quedado con ganas de más, algunos temas interesantes que no vimos y que pueden ser muy útiles:

- Excepciones (**`if`** y **`else`** dentro de bucles)
- Debugging: Capture, confirm y assert
- Formato de fechas
- Expresiones regulares
- Comando **`program`** para crear comandos personalizados